PYTHON CHEAT SHEET · 01 OF 05

Set & frozenset

Unordered collections of unique, hashable items: creating, updating, set algebra, frozenset, and the Student Course Enrollments program.

ORDERING	DUPLICATES	MUTABILITY	LOOKUP (IN)
Unordered, no indexing	**Never stored twice**	**set: yes · frozenset: no**	**O(1) on average**

01 Creating sets

```
>>> a = {1, 2, 3, 3, 2}                    # duplicates vanish
>>> a
{1, 2, 3}
>>> type(a)
<class 'set'>
>>> set([4, 5, 5, 6])                      # from any iterable
{4, 5, 6}
>>> sorted(set("mississippi"))             # unique characters (sorted for display)
['i', 'm', 'p', 's']
>>> set()                                  # the ONLY way to make an empty set
set()
>>> type({})                               # {} is an empty DICT, not a set
<class 'dict'>
>>> squares = {n * n for n in range(6)}    # set comprehension
>>> len(squares)
6
```

TIP Set elements must be **hashable** (immutable): numbers, strings, tuples of hashables and frozensets are fine; lists, dicts and sets are not.

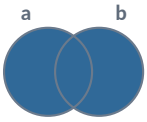
```
>>> {1, "two", (3, 4), 5.0}
{1, 'two', 5.0, (3, 4)}
>>> {[1, 2], 3}
TypeError: unhashable type: 'list'
```

02 Adding, removing and updating

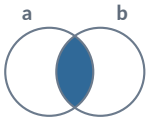
Task	Syntax	Notes
Add one item	`s.add(x)`	No effect if x is already present
Add many items	`s.update(iterable)`	Same as `s |= other`; accepts several iterables
Remove (strict)	`s.remove(x)`	Raises `KeyError` if x is missing
Remove (safe)	`s.discard(x)`	Silent when x is missing
Remove any item	`s.pop()`	Returns an arbitrary element; `KeyError` if the set is empty
Remove everything	`s.clear()`	Leaves `set()`
Shallow copy	`s.copy()`	Also `set(s)`
Size / membership	`len(s)` · `x in s`	`in` is a hash lookup, not a scan

```
>>> s = {10, 20, 30}
>>> s.add(40)
>>> s.add(10)                              # duplicate ignored
>>> s.update([50, 60], {70})
>>> s
{70, 40, 10, 50, 20, 60, 30}
>>> s.remove(20)
>>> s.discard(999)                         # missing item: no error
>>> s.remove(999)
KeyError: 999
>>> removed = s.pop()                      # arbitrary element: never rely on which one
>>> len(s)
5
```

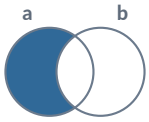
03 Set operations (set algebra)



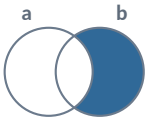
a | b
Union



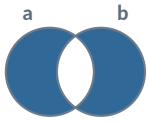
a & b
Intersection



a - b
Difference



b - a
Difference



a ^ b
Symmetric difference

Operation	Operator	Method	Result
Union	`a | b`	`a.union(b)`	Items in a or b (or both)
Intersection	`a & b`	`a.intersection(b)`	Items in both a and b
Difference	`a - b`	`a.difference(b)`	Items in a that are not in b
Symmetric difference	`a ^ b`	`a.symmetric_difference(b)`	Items in exactly one of the two
Subset	`a <= b`	`a.issubset(b)`	Every item of a is in b (`a < b` = proper subset)
Superset	`a >= b`	`a.issuperset(b)`	a contains every item of b (`a > b` = proper)
Disjoint	—	`a.isdisjoint(b)`	No items in common

In-place update	Operator	Method
Keep the union	`a |= b`	`a.update(b)`
Keep the intersection	`a &= b`	`a.intersection_update(b)`
Remove b's items	`a -= b`	`a.difference_update(b)`
Keep the symmetric difference	`a ^= b`	`a.symmetric_difference_update(b)`

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5, 6}
>>> a | b
{1, 2, 3, 4, 5, 6}
>>> a & b
{3, 4}
>>> a - b
{1, 2}
>>> b - a
{5, 6}
>>> a ^ b
{1, 2, 5, 6}
```

```
>>> a.union(b) == a | b                    # operator and method forms agree
True
>>> {3, 4} <= a                            # subset test
True
>>> {1, 2}.isdisjoint({3, 4})
True
>>> a.intersection([2, 3, 9])              # METHODS accept any iterable
{2, 3}
>>> a | [9]
TypeError: unsupported operand type(s) for |: 'set' and 'list'
```

TIP Operators need a set (or frozenset) on **both** sides; the named methods accept any iterable, e.g. `a.union([7, 8])`.

04 Handy patterns

```
>>> nums = [3, 1, 3, 2, 1]
>>> set(nums)                              # remove duplicates (order is lost)
{1, 2, 3}
>>> list(dict.fromkeys(nums))              # remove duplicates AND keep order
[3, 1, 2]
>>> len(set(nums))                         # how many distinct values?
3
>>> sorted({5, 2, 9}, reverse=True)        # sets are unordered: sort to get a list
[9, 5, 2]
>>> min({5, 2, 9}), max({5, 2, 9}), sum({5, 2, 9})
(2, 9, 16)
>>> any(x > 8 for x in {5, 2, 9})
True
>>> {x for x in range(10) if x % 3 == 0}   # filtered comprehension
{0, 9, 3, 6}
>>> {1, 1.0, True}                         # equal values with equal hashes collapse to one
{1}
```

05 frozenset: the immutable set

A `frozenset` supports every read-only set operation but none of the mutating ones. Because it is immutable it is **hashable**, so it can be a dictionary key or an element of another set.

	set	frozenset
Create	{1, 2} · set([1, 2])	frozenset([1, 2]) (no literal)
Mutable (add, remove, update, pop, clear)	Yes	No: raises `AttributeError`
Hashable (dict key / set element)	No	Yes
Union, intersection, difference, subset tests	Yes	Yes
Result type of `x | y`	Type of the left operand	Type of the left operand
Use it for	A working collection that changes	Constants, dict keys, safe sharing

```
>>> fs = frozenset([1, 2, 3, 2])
>>> fs
frozenset({1, 2, 3})
>>> fs.add(4)
AttributeError: 'frozenset' object has no attribute 'add'
>>> fs | {5}                              # left operand is a frozenset -> frozenset
frozenset({1, 2, 3, 5})
>>> {5} | fs                              # left operand is a set -> set
{1, 2, 3, 5}
>>> frozenset({1, 2}) == {1, 2}           # equality ignores the set/frozenset difference
True
>>> roles = {frozenset({"read"}): "viewer", frozenset({"read", "write"}): "editor"}
>>> roles[frozenset({"write", "read"})]   # frozenset as a dictionary key
'editor'
>>> len({frozenset({1, 2}), frozenset({2, 1})})   # a set of frozensets (sets of sets)
1
>>> {{1, 2}, {3}}
TypeError: unhashable type: 'set'
```

06 Complexity and gotchas

Operation	Average cost	Operation	Average cost
`x in s`	O(1)	`a | b`	O(len(a) + len(b))
`s.add(x) / s.remove(x)`	O(1)	`a & b`	O(min(len(a), len(b)))
`len(s)`	O(1)	`a - b`	O(len(a))

- `{}` creates an empty **dict**. Use `set()` for an empty set.
- Sets are unordered: don't rely on iteration order or on which item `pop()` returns.
- `set("abc")` splits the string into characters; use `{"abc"}` for a one-string set.
- Never add or remove items while looping over the same set (`RuntimeError: Set changed size during iteration`). Loop over `list(s)` or `s.copy()` instead.
- `1`, `1.0` and `True` are equal, so a set keeps only the first one it sees.

07 Sample program: managing student course enrollments

A training centre runs two student groups. Use sets to see which courses each group can take, which are shared, which are unique, whether a given course is offered, and which courses are mandatory and must never be altered (a `frozenset`).

#	Task	Concept shown
1	Create `group_a_courses` and `group_b_courses`, then print both	Set creation
2	Print the union of the two groups	`|` or `.union()`
3	Print the courses common to both groups	`&` or `.intersection()`
4	Print the courses in Group A but not in Group B	`-` or `.difference()`
5	Check whether "GitHub" is available in Group A	Membership with `in`
6	Define and print the `mandatory_courses` frozenset	`frozenset`

NOTE Set order can differ from run to run and between machines (string hashing is randomised), so your printed order may not match the output above. The members are always the same.

course_enrollments.py

```python
# Student course enrollments: sets and frozenset

# 1. Create the two course sets and print them
group_a_courses = {"Python", "SQL", "Power BI", "Azure"}
group_b_courses = {"Python", "Excel", "Power BI", "GitHub"}
print("Group A courses:", group_a_courses)
print("Group B courses:", group_b_courses)

# 2. Union: every course offered to either group
print("All courses (union):", group_a_courses | group_b_courses)
# same result: group_a_courses.union(group_b_courses)

# 3. Intersection: courses available to both groups
print("Common courses (intersection):", group_a_courses & group_b_courses)
# same result: group_a_courses.intersection(group_b_courses)

# 4. Difference: courses only Group A has
print("Only in Group A (difference):", group_a_courses - group_b_courses)
# same result: group_a_courses.difference(group_b_courses)

# 5. Membership test
print("Is GitHub available in Group A?", "GitHub" in group_a_courses)

# 6. Mandatory courses that must never change
mandatory_courses = frozenset({"Python", "SQL", "GitHub"})
print("Mandatory courses:", mandatory_courses)
```

Output

```
Group A courses: {'Azure', 'Power BI', 'Python', 'SQL'}
Group B courses: {'Excel', 'GitHub', 'Power BI', 'Python'}
All courses (union): {'Power BI', 'SQL', 'Excel', 'Azure', 'GitHub', 'Python'}
Common courses (intersection): {'Power BI', 'Python'}
Only in Group A (difference): {'Azure', 'SQL'}
Is GitHub available in Group A? False
Mandatory courses: frozenset({'GitHub', 'SQL', 'Python'})
```

Bonus: go one step further

course_enrollments_bonus.py

```python
# Continues the program above

# Courses unique to exactly ONE group (symmetric difference)
print("Unique to one group:", group_a_courses ^ group_b_courses)
print("Only in Group B:", group_b_courses.difference(group_a_courses))

# Which mandatory courses is Group A still missing?
print("Group A is missing:", mandatory_courses - group_a_courses)
print("Group A covers all mandatory courses?", mandatory_courses <= group_a_courses)

# A frozenset cannot be modified
try:
    mandatory_courses.add("Azure")
except AttributeError as err:
    print("Cannot modify a frozenset ->", err)
```

Output

```
Unique to one group: {'SQL', 'Excel', 'Azure', 'GitHub'}
Only in Group B: {'Excel', 'GitHub'}
Group A is missing: frozenset({'GitHub'})
Group A covers all mandatory courses? False
Cannot modify a frozenset -> 'frozenset' object has no attribute 'add'
```